🧠 L02: Variables

Memory, Storage & RAM

Neon City Cyber Academy

Module 1: Python Basics

👥 Your Team Recap

Emily Chen - *Handler*

> "Today we're diving into the computer's memory. Think of it as a giant locker room."

Cole Martinez - *Tech Lead*

> "Variables are how you label those lockers. Master this, and you control data."

🔄 Quick Review: print()

Last lesson you learned:

```
print("Hello, World!")
print(5 + 3)
print("=" * 20)
```

Problem: The data disappears after printing!

We need a way to **STORE** information for later use.

🧠 How Computer Memory Works

RAM (Random Access Memory)

Think of RAM as a **massive warehouse** with billions of storage boxes.

Each box has:

- 📍 An **address** (like 0x7FFF5E4C)
- 📦 **Contents** (the actual data)
- 🏷️ A **label** (optional - that's your variable!)

When you turn off the computer... **POOF!** RAM is erased.

💬 Emily Explains:

> "Imagine a locker room with millions of lockers. Each locker has a number (memory address)."

> "Without labels, you'd need to remember: 'My stuff is in locker #7,548,293,847'. IMPOSSIBLE!"

> "Variables are **stickers** you put on lockers so

📦 What is a Variable?

A **variable** is a named storage location in memory.

```
score = 100
```

Breaking it down:

- `score` → Variable name (the label)
- `=` → Assignment operator (NOT equals!)
- `100` → Value stored

Think: "Put 100 into the box labeled 'score'"

🎯 Creating Your First Variable

```
age = 14
print(age)      # Output: 14
```

What happened:

1. Python finds empty space in RAM
2. Stores the number `14`
3. Labels that spot as `age`
4. When you say `age`, Python fetches the value

Memory address might be `0x7FFF5E4C`, but you just say `age`!

✏️ Variable Naming Rules

✅ VALID names:

```
name = "Alex"
player_score = 100
level2 = True
_secret = "password"
```

❌ INVALID names:

```
2nd_place = "Silver"    # Can't start with number
my-score = 50           # No hyphens
class = "Python"        # 'class' is reserved
my score = 10           # No spaces
```

💬 Cole's Pro Tips:

> "Use **descriptive names**: `score` is better than `s`, `player_health` is better than `ph`."

> "Python convention: use `snake_case` (lowercase with underscores)."

> "Variables ARE case-sensitive: `Score` ≠ `score` ≠ `SCORE`"

🔄 Variables Can Change

That's why they're called **vari**ables!

```
health = 100
print(health)    # 100

health = 75      # Took damage!
print(health)    # 75

health = health - 25
print(health)    # 50
```

The old value gets **replaced** with the new one.

🧮 Using Variables in Math

```
attack = 10
defense = 5

damage = attack - defense
print(damage)    # 5
```

```
price = 50
quantity = 3

total = price * quantity
print(total)     # 150
```

Variables make calculations **readable and reusable!**

💬 Emily's Real Example:

> "Imagine a game:"

```
player_health = 100
enemy_damage = 23

player_health = player_health - enemy_damage
print("Health:", player_health)    # Health: 77
```

> "The computer READS the old value (100), subtracts 23, then WRITES back 77 into the same box!"

📊 Multiple Variables

You can create as many as you need:

```
name = "Shadow"
level = 5
experience = 1250
gold = 300
health = 85
mana = 120

print(name, "is level", level)
print("Gold:", gold)
```

Output:

```
Shadow is level 5
Gold: 300
```

🔁 Swapping Variables

Challenge: Swap two variables!

```
a = 10
b = 20
```

❌ This DOESN'T work:

```
a = b    # Now a=20, but old a (10) is lost!
b = a    # b=20 (both are 20 now!)
```

✅ Use a temporary box:

```
temp = a    # temp=10
a = b       # a=20
b = temp    # b=10
```

💬 Cole Breaks It Down:

> "Swapping is like switching drinks between two cups."

> "You can't pour cup A into cup B without spilling - you need a THIRD cup to hold one drink temporarily!"

```
cup_a = "coffee"
cup_b = "tea"
temp = cup_a          # temp holds coffee
cup_a = cup_b         # cup_a now has tea
cup_b = temp          # cup_b now has coffee
```

🆕 Updating Variables (Shortcuts)

The long way:

```
score = score + 10
health = health - 5
```

The shortcut:

```
score += 10     # Same as: score = score + 10
health -= 5     # Same as: health = health - 5
```

All shortcuts:

```
x += 5    # Add
x -= 3    # Subtract
x *= 2    # Multiply
x /= 4    # Divide
```

🎮 Building a Character Sheet

```
# Character creation
agent_name = "Nova"
level = 1
health = 100
attack = 15
defense = 10

#Display stats
print("=" * 30)
print("  AGENT PROFILE")
print("=" * 30)
print("Name:", agent_name)
print("Level:", level)
print("Health:", health)
print("Attack:", attack)
print("Defense:", defense)
print("=" * 30)
```

🔢 Memory Addresses (Advanced)

Want to see the ACTUAL memory address?

```
x = 42
print(id(x))    # Output: 140735621981234 (example)
```

`id()` shows the memory location in decimal.

In hexadecimal (how computers show it):

```
print(hex(id(x)))    # Output: 0x7fff5e4c1234
```

That `0x7F...` address is the **real locker number**!

💬 Emily's Mind-Blower:

> "Every piece of data in your computer - every number, every letter - lives at a memory address."

> "Variables just give those addresses human-readable names!"

```
player_score = 500
# Python: "0x7FF... holds 500, call it 'player_score'"

print(player_score)
# Python: "Go to 0x7FF..., grab the value, display it"
```

🐞 Common Mistakes

Error #1: Using before creating

```
print(score)    # NameError: name 'score' not defined
```

You must create it first!

Error #2: Typos

```
playerScore = 100
print(playerscore)    # Error! (case matters)
```

Error #3: Reserved words

```
print = 5    # Don't redefine Python keywords!
```

🧪 Challenge: Battle Simulator

```
hero_health = 100
monster_health = 80

hero_attack = 25
monster_attack = 15

# Round 1
monster_health -= hero_attack
print("Monster health:", monster_health)

hero_health -= monster_attack
print("Hero health:", hero_health)

# Who's winning?
```

Try it! Simulate 3 rounds of combat.

Project: Calculator with Memory

```
# Simple calculator
num1 = 25
num2 = 7

sum_result = num1 + num2
diff_result = num1 - num2
prod_result = num1 * num2
quot_result = num1 / num2

print("Numbers:", num1, "and", num2)
print("Sum:", sum_result)
print("Difference:", diff_result)
print("Product:", prod_result)
print("Quotient:", quot_result)
```

🎯 Challenge: Increment Game

```
counter = 0

counter += 1
print("Step 1:", counter)

counter += 1
print("Step 2:", counter)

counter += 5
print("Step 3:", counter)

counter -= 2
print("Step 4:", counter)

# What's the final value?
```

🔬 Advanced: Multiple Assignment

Python magic trick:

```
# Assign same value to multiple variables
a = b = c = 0

# Assign different values in one line
x, y, z = 10, 20, 30

print(x, y, z)    # 10 20 30
```

Swap without temp:

```
a, b = 5, 10
a, b = b, a    # Swap!
print(a, b)    # 10 5
```

💬 Cole's Parting Wisdom:

> "Variables are the FOUNDATION of programming."

> "Every app, game, and website you use has MILLIONS of variables storing data in RAM right now."

> "Master variables, and you're 50% of the way to being a real programmer!"

✅ Lesson Complete!

You Mastered:

- ✅ What RAM and memory addresses are
- ✅ Creating and naming variables
- ✅ Storing and changing values
- ✅ Using variables in calculations
- ✅ Update shortcuts (+=, -=, etc.)
- ✅ Seeing actual memory addresses with id()

Next Mission: L03 - Strings

The Index Train: Accessing characters one by one

🏆 Your Assignment

Create a **character stats tracker:**

1. Create 5 variables (name, level, health, mana, gold)
2. Display them in a formatted profile
3. Simulate gaining experience: `exp += 250`
4. Simulate taking damage: `health -= 30`
5. Print updated stats

Bonus: Use `+=` shortcuts for everything!

See You Next Lesson!

Remember: Variables are labeled boxes in computer memory.

You now control the **RAM**!